COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	July 16, 2026	**DATE SUBMITTED:**	July 16, 2026
NAME:	Benj Federic C. De Leon	**INSTRUCTOR:**	Engr. Jimlord M. Quejadoimlord

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays.

INPUT:

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
        std::cout << "Destructor Called." << std::endl;
}

    //Copy Constructor
Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
}

    //Display Attributes
```

```
void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
}

};

int main() {
Student student1("Roman", 28);
Student student2(student1);
Student student3;
student3 = student2;
return 0;

    return 0;
}
```

OUTPUT:

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Code Analysis:

The program reads main() from top to bottom, sequentially constructing the name and age for **student1** (Constructor Called.), then cloning those attributes into **student2** (Copy Constructor Called), and finally building **student3** using default data (Constructor Called.). After the line student3 = student2; quietly overwrites data without creating a new object, the program finishes reading main() and must clear the memory stack. it does not deconstruct them in the order they were made, but instead deconstructs **student3 first, student2 second, and student1 last**.

Table 2-1. Initial Driver Program

INPUT:

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
```

```
Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
        std::cout << "Destructor Called." << std::endl;
}

    //Copy Constructor
Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
}

    //Display Attributes
void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
}

};

int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

OUTPUT:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Code Analysis:

The program starts by executing main() sequentially. It initializes your constant size variable j to 5. Then, the program reads your studentList array declaration. It immediately moves to the first index (index 0) and runs the constructor. After getting all the initial info, it prints Constructor Called. While the program is running the constructors, it defaults the attributes inside the studentList to **"John Doe"** and **18**. Right after creating the students, it reads and sets up your namesList and ageList arrays on the stack. After executing everything inside main(), the program reaches return 0;. It must now clear all variables off the memory stack. It targets the 5 students and executes the destructors 5 consecutive times.

Table 2-2. Modified Driver Program with Student Lists

INPUT:

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
        std::cout << "Destructor Called." << std::endl;
}

    //Copy Constructor
```

```
Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
}

    //Display Attributes
void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
}

};

int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};
for(int i = 0; i < j; i++){ //loop A
Student *ptr = new Student(namesList[i], ageList[i]);
studentList[i] = *ptr;
}
for(int i = 0; i < j; i++){ //loop B
studentList[i].printDetails();
}
return 0;
}
```

OUTPUT:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
```

```
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Code Analysis:

In **Loop A**, the program runs five times and uses the word **new** to build a temporary, custom student out in extra storage space (the heap) for each name and age in your lists, which prints Constructor Called. five extra times. It then copies that new information directly into the blank student already waiting on your main table (the stack), but because the code doesn't use the word delete, these temporary storage students are left behind as a memory leak. Right after that, **Loop B** runs five times to read your main table and successfully prints out the updated names and ages (like "Carly 15" and "Freddy 16") on your screen just before the program closes and cleans up the table.

Table 2-3. Final Driver Program

INPUT (Modified):

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
        std::cout << "Destructor Called." << std::endl;
}

    //Copy Constructor
Student(const Student &copyStudent){
```

```
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
}

    //Display Attributes
void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
}

};

int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};
for(int i = 0; i < j; i++){ //loop A
    // Directly overwrite the stack student with the custom name and age
    studentList[i] = Student(namesList[i], ageList[i]);
}
for(int i = 0; i < j; i++){ //loop B
studentList[i].printDetails();
}
return 0;
}
```

OUTPUT (Modified):

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Code Analysis:

By changing Loop A to avoid the new keyword, the code prevents objects from being abandoned in heap storage, allowing C++ to automatically create a temporary student on the stack, copy its data into the array, and delete it immediately.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Summarization, checking if my idea is correct, explaining concepts, and code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: First I explain my concept idea then I let AI to clarify it.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.